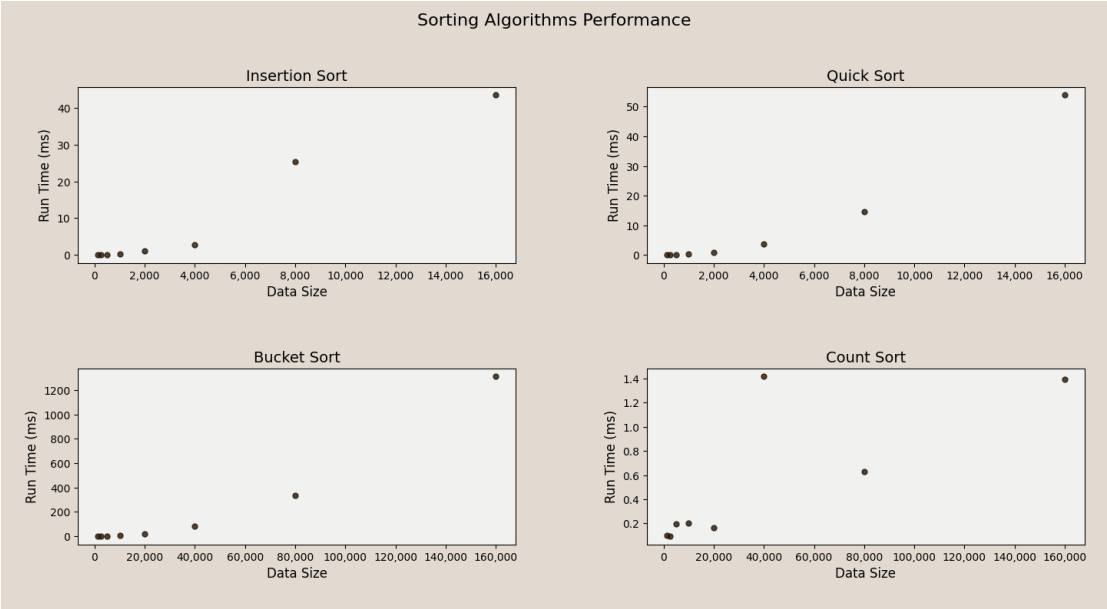
DSC 30 - PA 5

due Wednesday, 5/8/24 at 11:59PM

Problem 1.

Solution:

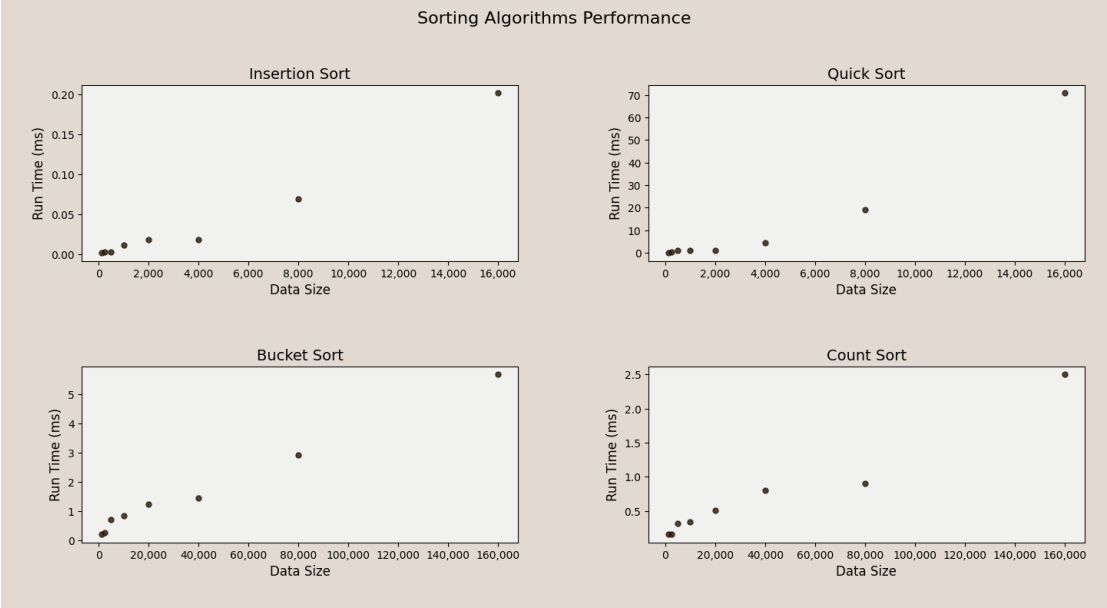


1. Insertion Sort: The time complexity that should be observed for the average case is $O(n^2)$. The plot seems to be $O(n^2)$ for the most part except for the bump in the middle of the graph. Considering the scale of the axes, it takes about as long as Quick sort, but is slightly faster.
2. Quick Sort: The time complexity that should be observed for the average case is $O(n \log(n))$. The plot seems to be $O(n \log(n))$ for the most part with uniform curvature throughout the graph. Considering the scale of the axes, it takes about as long as Insertion sort, but is slightly slower.
3. Bucket Sort: The time complexity that should be observed for the average case is $O(n + k)$. The plot seems to be have some curvature so I wouldn't say that it looks $O(n)$. In fact, it looks most similar to Quick sort. Considering it does the same data sizes as Count sort, it's much, much slower than Count sort.
4. Count Sort: The time complexity that should be observed for the average case is $O(n + k)$. The plot seems to be $O(n)$ for the most part except for the random bumps in the graph. Considering it does the same data sizes as Bucket sort, it's about 1000 times faster.

Problem 2.

Solution:

```java
public static ArrayList<Integer> generateArrayList(int size) {
    ArrayList<Integer> arr = new ArrayList<>();
    for (int i = 0; i < size; i++) {
        arr.add(i);
    }
    return arr;
}
```

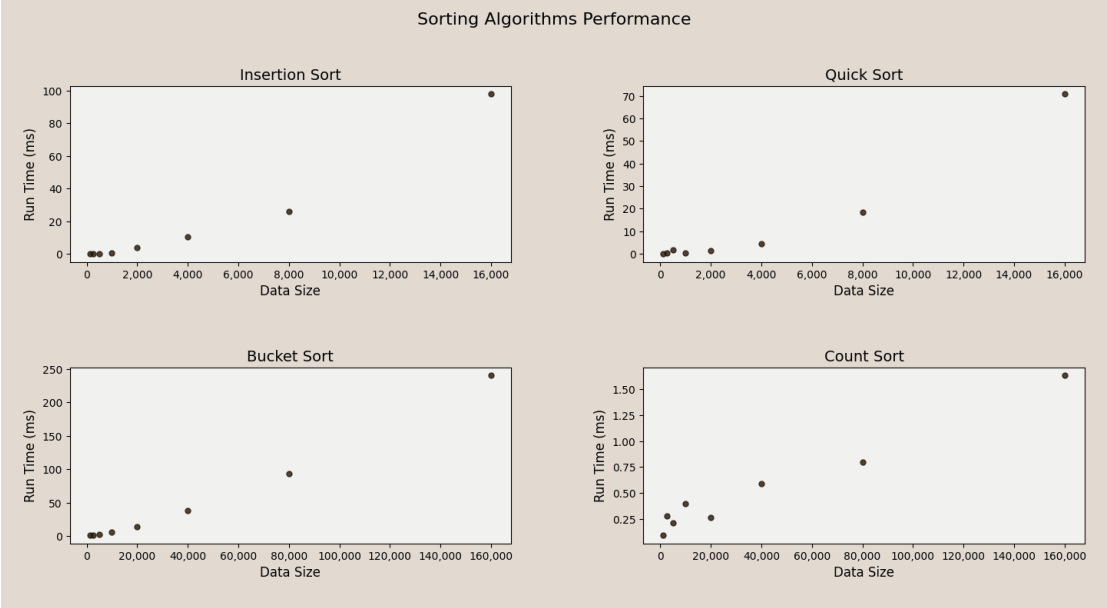


By modifying generateArrayList() to create sorted arrays, we're able to achieve the best case scenario for Insertion sort which is O(n) since the inner while loop doesn't have to run anymore. This cut down it's runtime by over an order of magnitude. In addition, note that Bucket sort's runtime was also cut down by over 2 orders of magnitude as well and is now running only slightly slower than count sort.

Problem 3.

Solution:

```java
public static ArrayList<Integer> generateArrayList(int size) {
    ArrayList<Integer> arr = new ArrayList<>();
    for (int i = 1; i <= size; i++) {
        arr.add(size-i);
    }
    return arr;
}
```

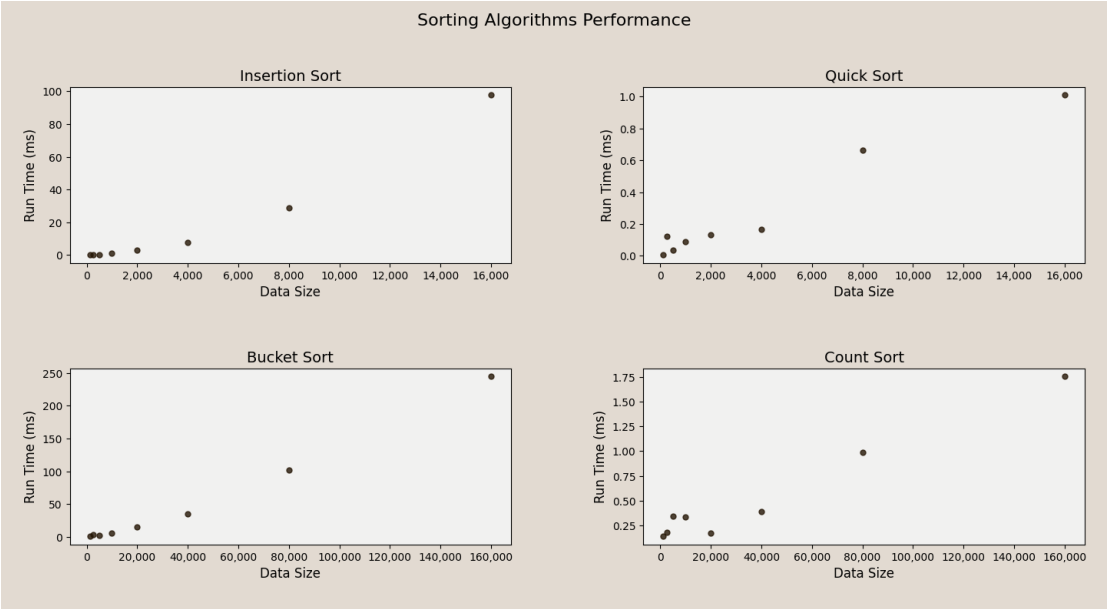


By modifying generateArrayList() to create reverse arrays, we're able to achieve the worst case scenario for Quick sort which is $O(n^2)$ since n each partition only moves the first element to the end. This makes it no different from selection sort. However, this only seems to increase the runtime slightly, taking 70 ms vs 50 ms for a data size of 16,000.

Problem 4.

Solution:

```java
private static int inPlacePartition(ArrayList<Integer> arr, int start, int stop, int pivotIx) {
    int pivot = arr.get((start + stop) / 2);
    swap(arr, pivotIx, stop);
    int middleBarrier = start;
    for (int endBarrier = start; endBarrier < stop; endBarrier++) {
        if (arr.get(endBarrier) < pivot) {
            swap(arr, middleBarrier, endBarrier);
            middleBarrier++;
        }
    }
    swap(arr, middleBarrier, stop);
    return middleBarrier;
}
```

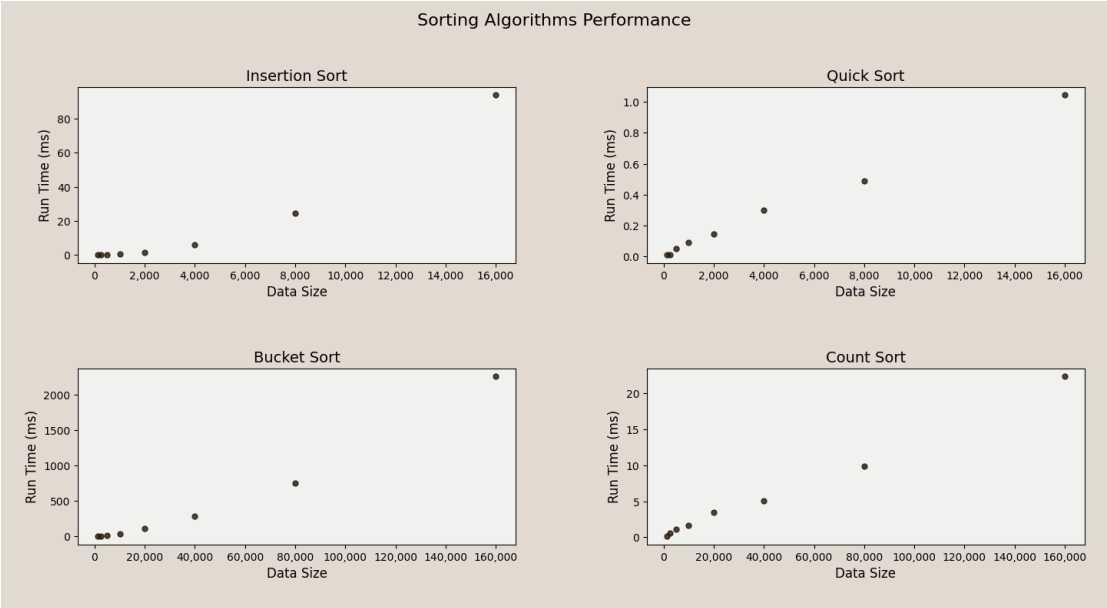


After modifying inPlacePartition() to use the middle element as the pivot while keeping the reverse arrays in generateArrayList(), Quick sort became much faster and is even resembling O(n) time complexity. It's runtime has dropped by two orders of magnitude as well.

Problem 5.

Solution:

```java
public static ArrayList<Integer> generateArrayList(int size) {
    ArrayList<Integer> arr = new ArrayList<>();
    arr.add(size*100);
    for (int i = 1; i < size; i++) {
        arr.add(size-i);
    }
    return arr;
}
```

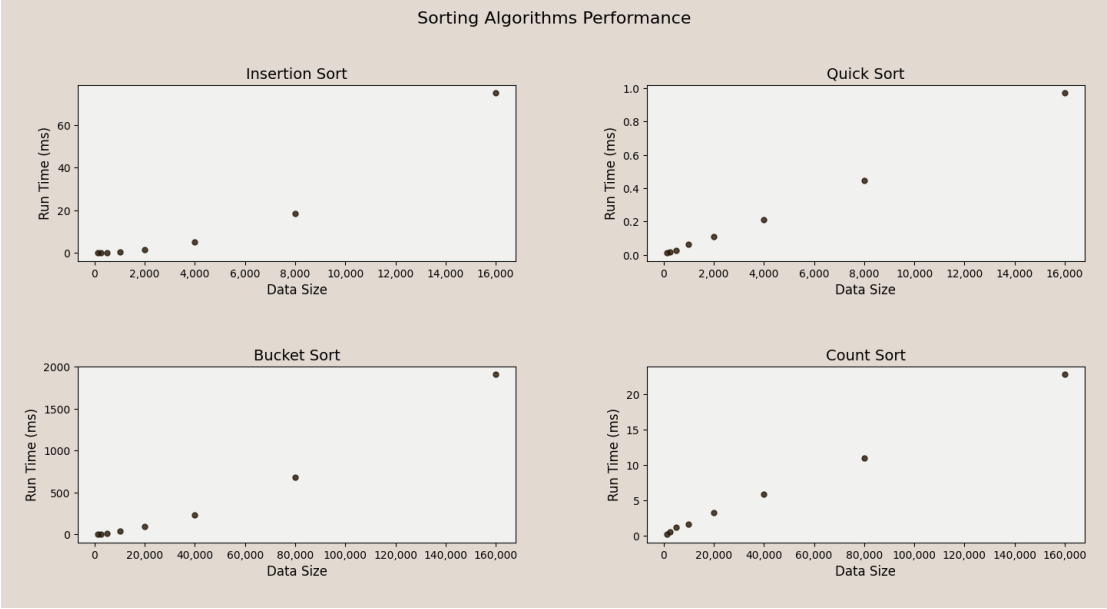


By modifying generateArrayList() to generate an array in reverse order with the first element being 100 times n, we're able to slow down Bucket sort considerably. This is because bucket sort will place all but the first element into the same bucket and then insertion sort will have the worst case scenario of $O(n^2)$ when sorting that bucket since the elements are in reverse order. This means that Bucket sort will also have a time complexity of $O(n^2)$ and is why it took almost twice as long as before.

Problem 6.

Solution:

```java
public static ArrayList<Integer> generateArrayList(int size) {
    ArrayList<Integer> arr = new ArrayList<>();
    arr.add(size*100);
    for (int i = 1; i < size; i++) {
        arr.add(size-i);
    }
    return arr;
}
```



By modifying generateArrayList() to generate an array in reverse order with the first element being 100 times n, we're able to slow down Count sort considerably. This is because setting up the count array will take a time complexity of $100n$ which is still $O(n)$. However, you can slow down count sort even more if the data's range is n^2 rather than $100n$. I would've done this, but I ran out of memory in the heap so I toned it down to only $100n$ so my computer could run it. However, even this is enough to slow down count sort by 20 times.